

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Buisness Analytics Practical – VIII

Roll No :-	Name :-
Class :- TYIT	Batch :-
Date of Assignment :-11/09/2026	Date/Time of Submission :- 11/09/2026

8. Confidence Interval Estimation:

A) Calculate confidence intervals for population means or proportions using sample data and interpret the results in a business context.

Load the supermarket_sales.csv dataset into a Pandas DataFrame.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
df = pd.read_csv("supermarket_sales.csv")
print("Dataset Loaded Successfully")
```

✓ 0.0s

Dataset Loaded Successfully

Display the first five records and check the size of the dataset.

```
print("\nFirst Five Records:")
print(df.head())
print("\nSize of Dataset:")
print(df.shape)
print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
```

✓ 0.0s

First Five Records:

	Invoice ID	Branch	City	Customer type	Gender
0	750-67-8428	A	Yangon	Member	Female
1	226-31-3081	C	Naypyitaw	Normal	Female
2	631-41-3108	A	Yangon	Normal	Male
3	123-19-1176	A	Yangon	Member	Male
4	373-73-7910	A	Yangon	Normal	Male

	Product line	Unit price	Quantity	Tax 5%	Total	Date
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019
4	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019

	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	10:29	Cash	76.40	4.761905	3.8200	9.6
2	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	20:33	Ewallet	465.76	4.761905	23.2880	8.4
4	10:37	Ewallet	604.17	4.761905	30.2085	5.3

Size of Dataset:
(1000, 17)
Number of Rows: 1000
Number of Columns: 17

Select the revenue column for analysis.

```
df["revenue"] = df["Total"]
revenue = df["revenue"]
```

✓ 0.0s

```
print("\nMissing Values in Revenue:", revenue.isnull().sum())
```

✓ 0.0s

Missing Values in Revenue: 0

```
revenue = revenue.dropna()
print("Missing values handled successfully")
```

✓ 0.0s

Missing values handled successfully

```
n = len(revenue)
sample_mean = revenue.mean()
sample_std = revenue.std()

print("\nSample Size:", n)
print("Sample Mean:", round(sample_mean, 2))
print("Sample Standard Deviation:", round(sample_std, 2))
```

✓ 0.0s

Sample Size: 1000
Sample Mean: 322.97
Sample Standard Deviation: 245.89

```
standard_error = stats.sem(revenue)

# 90% Confidence Interval
ci_90 = stats.t.interval(
    confidence=0.90,
    df=n - 1,
    loc=sample_mean,
    scale=standard_error
)
# 95% Confidence Interval
ci_95 = stats.t.interval(
    confidence=0.95,
    df=n - 1,
    loc=sample_mean,
    scale=standard_error
)
# 99% Confidence Interval
ci_99 = stats.t.interval(
    confidence=0.99,
    df=n - 1,
    loc=sample_mean,
    scale=standard_error
)
```

✓ 0.0s

```
ci_table = pd.DataFrame({
    "Confidence Level": ["90%", "95%", "99%"],
    "Lower Limit": [
        ci_90[0],
        ci_95[0],
        ci_99[0]
    ],
    "Upper Limit": [
        ci_90[1],
        ci_95[1],
        ci_99[1]
    ]
})

print("\nConfidence Intervals:")
print(ci_table.round(2))
```

✓ 0.0s

Confidence Intervals:

	Confidence Level	Lower Limit	Upper Limit
0	90%	310.17	335.77
1	95%	307.71	338.23
2	99%	302.90	343.03

```

print("\nComparison:")
print("90% CI :", round(ci_90[0], 2), "to", round(ci_90[1], 2))
print("95% CI :", round(ci_95[0], 2), "to", round(ci_95[1], 2))
print("99% CI :", round(ci_99[0], 2), "to", round(ci_99[1], 2))
print("\nAs confidence level increases,")
print("the confidence interval becomes wider.")

```

✓ 0.0s

Comparison:

90% CI : 310.17 to 335.77

95% CI : 307.71 to 338.23

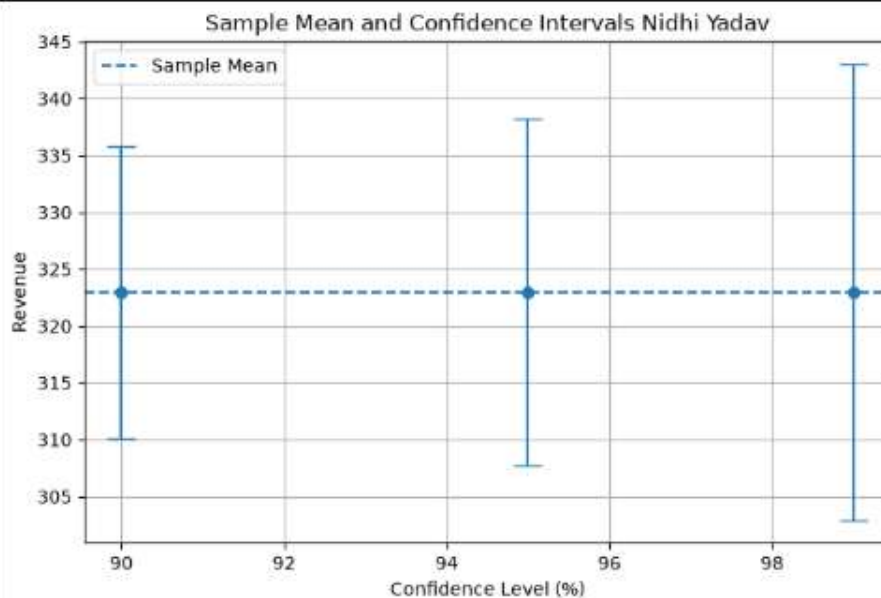
99% CI : 302.9 to 343.03

As confidence level increases,
the confidence interval becomes wider.

```

confidence_levels = [90, 95, 99]
lower_values = [
    ci_90[0],
    ci_95[0],
    ci_99[0]
]
upper_values = [
    ci_90[1],
    ci_95[1],
    ci_99[1]
]
errors_lower = [
    sample_mean - ci_90[0],
    sample_mean - ci_95[0],
    sample_mean - ci_99[0]
]
errors_upper = [
    ci_90[1] - sample_mean,
    ci_95[1] - sample_mean,
    ci_99[1] - sample_mean
]
plt.figure(figsize=(8, 5))
plt.errorbar(
    confidence_levels,
    [sample_mean] * 3,
    yerr=[errors_lower, errors_upper],
    fmt='o',
    capsize=8
)
plt.axhline(
    sample_mean,
    linestyle='--',
    label="Sample Mean"
)
plt.xlabel("Confidence Level (%)")
plt.ylabel("Revenue")
plt.title("Sample Mean and Confidence Intervals Nidhi Yadav")
plt.legend()
plt.grid(True)
plt.show()

```



```

print("\nInterpretation of 95% Confidence Interval:")
print(
    "We are 95% confident that the true population mean "
    "revenue lies between",
    round(ci_95[0], 2),
    "and",
    round(ci_95[1], 2)
)

```

✓ 0.0s

Interpretation of 95% Confidence Interval:

We are 95% confident that the true population mean revenue lies between 307.71 and 338.23

```

customer_type = df["Customer type"]

print("\nCustomer Type Counts:")
print(customer_type.value_counts())

print("\nCustomer Type Proportions:")
proportions = customer_type.value_counts(normalize=True)

print(proportions)

```

✓ 0.0s

Customer Type Counts:

Customer type

Member 501

Normal 499

Name: count, dtype: int64

Customer Type Proportions:

Customer type

Member 0.501

Normal 0.499

Name: proportion, dtype: float64

```

total_customers = len(customer_type)

member_count = (customer_type == "Member").sum()

member_proportion = member_count / total_customers

# Standard error for proportion
proportion_se = np.sqrt(
    member_proportion * (1 - member_proportion)
    / total_customers
)

# 95% confidence interval using normal approximation
z = stats.norm.ppf(0.975)

proportion_lower = member_proportion - z * proportion_se
proportion_upper = member_proportion + z * proportion_se

# Keep limits between 0 and 1
proportion_lower = max(0, proportion_lower)
proportion_upper = min(1, proportion_upper)

print("\nMember Customers:", member_count)
print("Total Customers:", total_customers)
print("Member Proportion:", round(member_proportion, 4))

print("\n95% Confidence Interval for Member Proportion:")
print(
    round(proportion_lower, 4),
    "to",
    round(proportion_upper, 4)
)

print(
    "Percentage:",
    round(proportion_lower * 100, 2),
    "% to",
    round(proportion_upper * 100, 2),
    "%"
)

```

✓ 0.0s

Member Customers: 501
 Total Customers: 1000
 Member Proportion: 0.501

95% Confidence Interval for Member Proportion:
 0.47 to 0.532
 Percentage: 47.0 % to 53.2 %

```
print("\nBusiness Interpretation:")
print(
    "We are 95% confident that the true proportion "
    "of Member customers lies between",
    round(proportion_lower * 100, 2),
    "% and",
    round(proportion_upper * 100, 2),
    "%"
)
```

✓ 0.0s

Business Interpretation:
 We are 95% confident that the true proportion of Member customers lies between 47.0 % and 53.2 %

B) Apply bootstrapping techniques to estimate confidence intervals for non-parametric statistics.

Part B

```
original_median = revenue.median()
print("\nOriginal Sample Median:", round(original_median, 2))
```

✓ 0.0s

Original Sample Median: 253.85

```
np.random.seed(42)

bootstrap_medians = []

# 1000 bootstrap samples
for i in range(1000):

    # 17. Create bootstrap sample
    bootstrap_sample = np.random.choice(
        revenue,
        size=len(revenue),
        replace=True
    )

    # 18. Calculate median
    bootstrap_median = np.median(bootstrap_sample)

    # 19. Store median
    bootstrap_medians.append(bootstrap_median)

# Convert to NumPy array
bootstrap_medians = np.array(bootstrap_medians)

print("\nNumber of Bootstrap Samples:",
      len(bootstrap_medians))
```

✓ 0.0s

Number of Bootstrap Samples: 1000

```
lower_percentile = np.percentile(
    bootstrap_medians,
    2.5
)

upper_percentile = np.percentile(
    bootstrap_medians,
    97.5
)

print("\n2.5th Percentile:",
      round(lower_percentile, 2))

print("97.5th Percentile:",
      round(upper_percentile, 2))
```

✓ 0.0s

2.5th Percentile: 232.87
 97.5th Percentile: 273.61

```
bootstrap_ci = (
    lower_percentile,
    upper_percentile
)

print("\n95% Bootstrap Confidence Interval:")
print(
    round(bootstrap_ci[0], 2),
    "to",
    round(bootstrap_ci[1], 2)
)
```

✓ 0.0s

95% Bootstrap Confidence Interval:
 232.87 to 273.61

Write a conclusion explaining how confidence intervals and bootstrapping can help businesses make decisions using sample data.

```
print("\nBusiness Interpretation:")
print(
    "The bootstrap confidence interval gives an estimated "
    "range for the true population median revenue."
)

print(
    "The estimated population median revenue is between",
    round(bootstrap_ci[0], 2),
    "and",
    round(bootstrap_ci[1], 2)
)
} ✓ 0.0s

Business Interpretation:
The bootstrap confidence interval gives an estimated range for the true population median revenue.
The estimated population median revenue is between 232.87 and 273.61

print("\nCONCLUSION:")
print(
    "Confidence intervals help businesses estimate the "
    "population mean and customer proportions from sample data."
)

print(
    "Bootstrapping helps estimate the confidence interval "
    "of statistics such as the median without assuming "
    "a specific data distribution."
)

print(
    "These methods help businesses make better decisions "
    "using sample data."
)
} ✓ 0.0s

CONCLUSION:
Confidence intervals help businesses estimate the population mean and customer proportions from sample data.
Bootstrapping helps estimate the confidence interval of statistics such as the median without assuming a specific data distribution.
These methods help businesses make better decisions using sample data.
```

Justification

The given code is justified because it uses **Confidence Interval Estimation and Bootstrapping** on the supermarket_sales.csv dataset to estimate important population values from sample data. The Total column is used as revenue, and missing values are removed before calculating the sample mean, standard deviation, and confidence intervals. The code calculates **90%, 95%, and 99% confidence intervals** to compare how the interval becomes wider as the confidence level increases.

It also calculates the **proportion of Member customers** and finds its 95% confidence interval, which helps understand the percentage of customers who are members. Finally, bootstrapping with **1000 samples** is used to estimate the confidence interval of the median revenue without assuming a particular distribution. The graphs help visualize the confidence intervals and bootstrap median distribution.

Conclusion

The practical successfully demonstrates **confidence interval estimation and bootstrapping** using supermarket sales data. The code estimates the population mean revenue, Member customer proportion, and median revenue with appropriate confidence intervals. It shows that higher confidence levels produce wider intervals and that bootstrapping can be used to estimate the confidence interval of the median. These statistical methods help businesses understand their sales data and make better decisions based on sample information.